INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	**INTRODUCCIÓN AL DESARROLLO WEB**				
TITULO DE LA PRÁCTICA:	*Entorno de Ejecución Node.js, Gestión de Paquetes con npm y Creación de Servidor HTTP con Módulos Nativos*				
NÚMERO DE PRÁCTICA:	*05*	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*Segundo semestre*
FECHA DE PRESENTACIÓN	*19/10/2026*	**HORA DE PRESENTACIÓN**	*hh/mm/00*		
INTEGRANTE (s) *-Aldo Aron Cabana Perales* *-Andre Chacondori Mayta* *-Aldair Aguilar Quenta*				**NOTA (0-20)**	*Nota colocada por el docente*
DOCENTE(s): **MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.**					

RESULTADOS Y PRUEBAS

I. EJERCICIOS RESUELTOS:

El estudiante coloca la evidencia de los ejercicios propuestos realizados en la sesión de laboratorio, en el tiempo o duración indicado por el docente. Indicando

- *Colocar enunciados del ejercicio.*
- *Mostrar la solución mediante de capturas de pantalla y adjuntando la evidencia (archivos adjuntos) de acuerdo al ejercicio planteado.*
- *Mostrar código fuente al 100%.*
- *Indicar enlace o link de repositorio GITHUB.*

II. PROBLEMAS PROPUESTOS RESUELTOS:

El estudiante coloca la evidencia de los problemas propuestos realizados en la sesión de laboratorio, en el tiempo o duración indicado por el docente. Indicando:

- *Colocar enunciados del problema.*
- *Mostrar la solución mediante de capturas de pantalla y adjuntando la evidencia (archivos adjuntos) de acuerdo al problema planteado.*
- *Mostrar código fuente al 100%.*
- *Indicar enlace o link de repositorio GITHUB.*

Caso Práctico: Servidor Backend Estático y API Básica de Consultas Desarrollar una aplicación en Node.js desde cero implementando los siguientes requerimientos:

1. Configuración de Proyecto npm:

○ Inicializar el proyecto ejecutando npm init -y.

○ Configurar scripts en el package.json: "start": "node server.js" y "dev": "node --watch server.js".

○ Crear una estructura modular de carpetas: /public (archivos estáticos HTML/CSS), /data (archivo JSON con datos) y server.js.

a)

```
PS C:\Users\Alan CABANA\Desktop\Aldo-Git-Hub\IDWeb\Lab5> npm init -y
Wrote to C:\Users\Alan CABANA\Desktop\Aldo-Git-Hub\IDWeb\Lab5\package.json:

{
  "name": "lab5",
  "version": "1.0.0",
  "description": "Servidor HTTP estático + API básica de estudiantes",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

b)

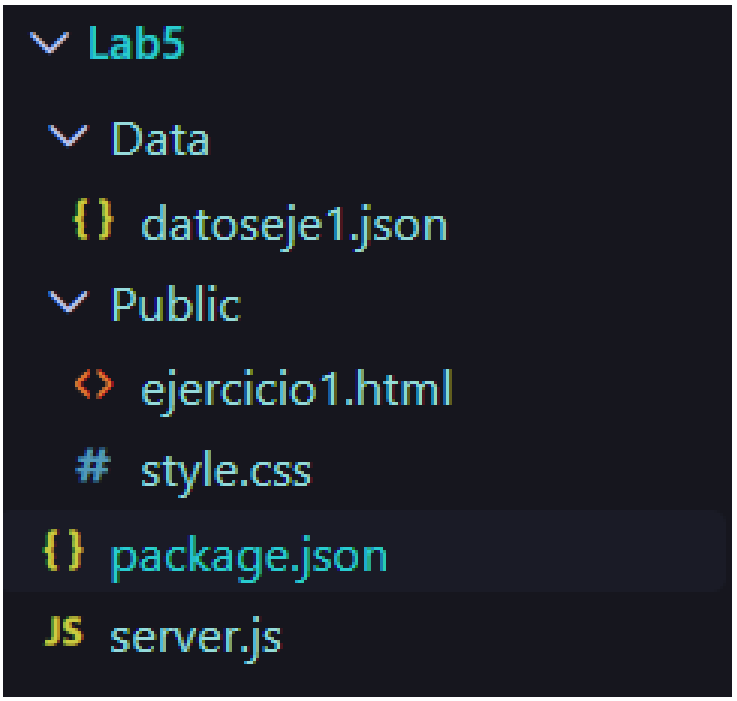
```
{
  "name": "lab5",
  "version": "1.0.0",
  "description": "Servidor HTTP estático + API básica de estudiantes",
  "main": "server.js",
  ▷Debug
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
PS C:\Users\Alan CABANA\Desktop\Aldo-Git-Hub\IDWeb\Lab5> npm run dev

> lab5@1.0.0 dev
> node --watch server.js

Servidor listo para programar
Completed running 'server.js'. Waiting for file changes before restarting...
```

c)



2. Construcción del Servidor HTTP Nativo:
○ Instanciar el servidor mediante http.createServer escuchando en el puerto 3000.
○ Servir archivos estáticos desde la carpeta /public asíncronamente con fs.readFile infiriendo correctamente el Content-Type (ej. text/html, text/css).

```
const http = require ("http");
const fs = require("fs");
const path = require ("path");

const PORT = 3000;
const PUBLIC_DIR = path.join(__dirname, "public");
//CREA LA RUTA, HASTA LA CARPETA PUBLIC
const DATA = path.join(__dirname,"data" ,"estudiantes.json");
//CREA LA RUTA, HASTA LA CARPETA DATA Y LUEGO AL JS
//DIRNAME TE DA LA RUTA HASTA ESA CARPETA :)

const MINE_TYPES= {
  '.html':'text/html; charset=utf-8',
  '.css': 'text/css; charset=utf-8',
  '.js':  'text/javascript; charset=utf-8',
  '.json':'application/json; charset=utf-8',
  '.png': 'image/png',
  '.jpg': 'image/jpeg',
  '.svg': 'image/svg+xml',
  '.ico': 'image/x-icon'
};

function enviarJson (res, estat, obj) {
    res.writeHead (estat, {"Content-Type":"aplication/json; charset=uft-8"} )
    //ENVIA ESTADO CON UN BUEN FORMATO
    res.end(JSON.stringify(obj));
    //DEVUELVE EL OBJTEO EN TEXTO
}
```

```
function enviarJson (res, estat, obj) {
    res.writeHead (estat, {"Content-Type":"aplication/json; charset=uft-8"} )
    //ENVIA ESTADO CON UN BUEN FORMATO
    res.end(JSON.stringify(obj));
    //DEVUELVE EL OBJTEO EN TEXTO
}

const server =http.createServer ((req, res) =>{
const {pathname} = new URL (req.url , "http://localhost")

console.log (`Peticion recibida: ${req.url} ${pathname}`);

if (pathname=== '/api/estudiantes'&& req.method==="GET"){
 fs.readFile(DATA, 'utf-8', (err, contenido) => {
  // Error de lectura
  if (err) {
    res.writeHead(500, { 'Content-Type': 'application/json' });
    //manda mensaje de error en formato json :v
    return res.end(JSON.stringify({ error: 'No se pudo leer el archivo' }));
  }

  try {
    // JSON.parse intenta leer el texto. Sino, defrente al catch
    JSON.parse(contenido);
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(contenido);
  }
  //Manda el error
  catch (errorValidacion) {
    res.writeHead(400, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Formato JSON inválido' }));
  }
});
```

```
else if(pathname==='/api/estudiantes' && req.method==="POST" ) {
  // Fase 5: POST
  let body = '';
  //ALMACENA EVENTOS DATA
  req.on('data', chunk => {
      body += chunk.toString();
  });
  //USA PARA VERIFICAR ERRORES
  req.on('end', () => {
    let nuevoEstudiante;
    try {
      nuevoEstudiante = JSON.parse(body);
    } catch (error) {
      return enviarJson(res, 400, { error: 'Formato JSON inválido' });
    }

    if (typeof nuevoEstudiante !== 'object' || nuevoEstudiante === null || Array.isArray(nuevoEstudiante)) {
      return enviarJson(res, 400, { error: 'Se esperaba un objeto estudiante' });
    }

    fs.readFile(DATA, 'utf-8', (err, contenido) => {
      if (err) {
        return enviarJson(res, 500, { error: 'No se pudo leer el archivo' });
      }

      let estudiantes;
      try {
        estudiantes = JSON.parse(contenido);
        if (!Array.isArray(estudiantes)) throw new Error('No es un arreglo');
      } catch (errorParse) {
        return enviarJson(res, 500, { error: 'El archivo de datos tiene un formato inválido' });
      }

      estudiantes.push(nuevoEstudiante);
```

```
        fs.writeFile(DATA, JSON.stringify(estudiantes, null, 2), 'utf-8', (errEscritura) => {
          if (errEscritura) {
            return enviarJson(res, 500, { error: 'No se pudo guardar el estudiante' });
          }
          enviarJson(res, 201, nuevoEstudiante);
        });
      });
    });
}

else if( !pathname.startsWith('/api/') && req.method==="GET" ) {
  const archivoSol = pathname==="/" ? "index.html" :pathname ;
  const rutaArch = path.join(__dirname, "public", archivoSol);
  const contentType = MINE_TYPES[String(path.extname(rutaArch)).toLowerCase()] || 'application/octet-stream';

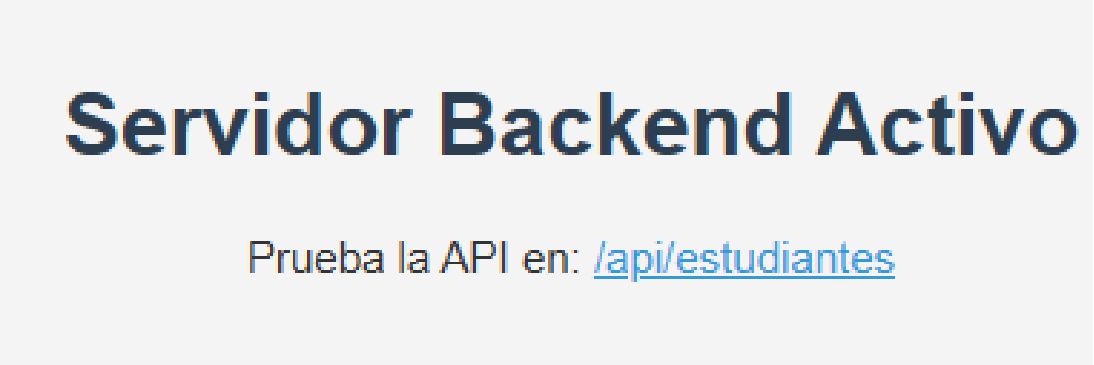

  fs.readFile(rutaArch, (err, contenido) => {
    if (err) {
      enviarJson(res, 404, { error: 'Archivo no encontrado', mensaje: 'El recurso no existe' });
      return;
    }
    res.writeHead(200, { 'Content-Type': contentType });
    res.end(contenido);

  });
}
else { enviarJson(res,404 ,{
  error: "not found",
  mensaje: `la ruta ${pathname}, con el metodo${req.method} NO EXISTE`

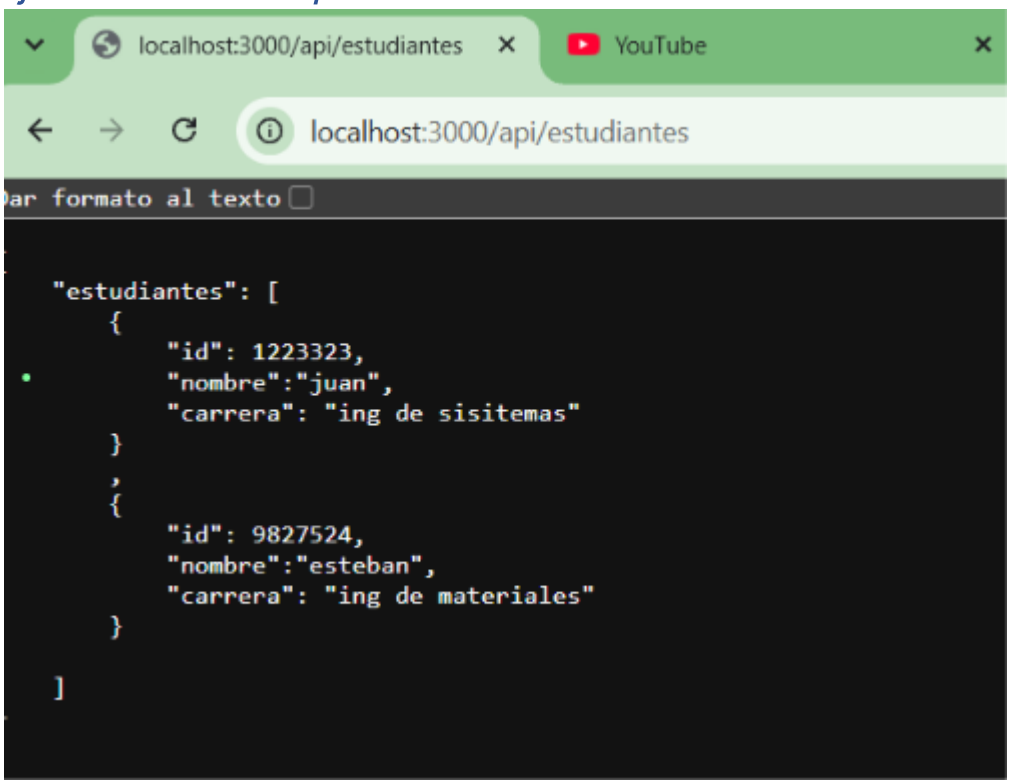
});
}
});
```

```
server.listen (PORT,() =>{
  console.log(`Servidor escuchando en http://localhost:${PORT}` );
});
```


http://localhost:3000/
Servidor Backend Activo
Prueba la API en: /api/estudiantes
Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\Alan CABANA\Desktop\Aldo-Git-Hub\IDWeb\Lab5> npm start
> lab5@1.0.0 start
> node server.js
Servidor en http://localhost:3000
Buscando el archivo en esta ruta: C:\Users\Alan CABANA\Desktop\Aldo-Git-Hub\IDWeb\Lab5\public\index.html
Buscando el archivo en esta ruta: C:\Users\Alan CABANA\Desktop\Aldo-Git-Hub\IDWeb\Lab5\public\style.css
node
node


Servidor Backend Activo
Prueba la API en: /api/estudiantes

Ejecución de ruta en dispositivo:



3. Implementación de Endpoints Basados en Rutas:

○ Ruta GET /api/estudiantes: Leer el archivo /data/estudiantes.json de forma asíncrona y retornar la lista de estudiantes en formato JSON con código 200 OK.
○ Ruta POST /api/estudiantes: Capturar los chunks de datos del body de la petición (evento req.on('data') y req.on('end')), parsear el objeto, agregarlo al archivo JSON de persistencia local y retornar el registro guardado con código 201 Created.
○ Manejo de rutas no existentes retornando un JSON con código 404 Not Found

CUESTIONARIO:

Colocar la evidencia de las respuestas realizadas al cuestionario enunciado en la guía práctica de laboratorio. Indicando:

-Colocar enunciados de la pregunta planteada.

-Indicar la respuesta y justificar su respuesta.

1. Explique la arquitectura orientada a eventos y el modelo de I/O No Bloqueante (Non-blocking I/O) de Node.js. ¿En qué se diferencia de servidores multihilo tradicionales como Apache Tomcat?

Node.js usa un solo hilo: cuando una tarea tarde (leer un archivo, esperar la red) no se queda esperando, la manda, “por detrás”, y sigue con otras, Asi atiende muchas conexiones con poca memoria. Tomcat abre un hilo por usuario: gasta mas memoria pero va mejor en cálculos pesados. Node es bueno con muchas esperas, pero se traba con cálculos pesados

2. ¿Qué función cumple la carpeta node_modules y por qué nunca debe incluirse en el control de versiones (Git)? ¿Qué archivo controla exactamente el árbol de dependencias?

Node_modules guarda los paquetes que necesita el proyecto. No se sube a Git porque se recrea con npm install, pesa mucho y desordena el historial. Package.json dice que paquetes quieres y package-lock.json guarda las versiones exactas; esos dos si se suben

3. ¿Cuál es la diferencia entre utilizar métodos síncronos de File System (fs.readFileSync) frente a métodos asíncronos (fs.readFile) en un servidor de producción?
readFileSync deja al servidor esperando hasta terminar de leer, sin atender a nadie mas; por eso no se usa en producción. Readfile lee “por detrás” y sigue atendiendo a los demas usuarios. En producción siempre la versión asíncrona; la síncrona solo al arrancar o en scripts pequeños

CONCLUSIONES

En esta práctica se comprendió el funcionamiento de Node.js al implementar un servidor HTTP nativo y una API REST básica con persistencia local en archivos JSON. Mediante el uso de la arquitectura basada en eventos y el manejo asíncrono de E/S, se logró gestionar de forma eficiente la entrega de archivos estáticos con sus respectivos tipos MIME, así como el procesamiento de peticiones GET y POST con sus correspondientes códigos de estado HTTP. Asimismo, se afianzó la correcta administración del entorno del proyecto y sus dependencias mediante el uso de npm y la configuración de package.json.

METODOLOGÍA DE TRABAJO

Para el desarrollo de la práctica se empleó una metodología colaborativa e incremental. El código fuente del servidor HTTP y los archivos del proyecto se gestionaron mediante un repositorio compartido en GitHub para el control de versiones, mientras que la redacción del informe de laboratorio y la consolidación de evidencias se realizaron de forma síncrona utilizando un documento en Google Docs / Word Online. En el ámbito técnico, se inicializó el entorno con npm, se estructuró el proyecto de forma modular (/public y /data) se validó el comportamiento del servidor localmente mediante scripts automatizados en la terminal de Visual Studio Code.

Node.js Foundation. (s. f.). *Node.js v20.x Documentation - File System & HTTP Modules*. Recuperado el 19 de octubre de 2026, de https://nodejs.org/docs/latest/api/

npm, Inc. (s. f.). *npm Documentation: package.json y gestión de dependencias*. Recuperado el 19 de octubre de 2026, de https://docs.npmjs.com/

MDN Web Developers. (2026). *Tipos MIME (IANA media types)*. Mozilla Developer Network. https://developer.mozilla.org/es/docs/Web/HTTP/Basics_of_HTTP/MIME_types